

Introduction to Blockchain and Smart Contracts

Lecture 4: Consensus

Subodh Sharma | IIT Delhi | Jan 15, 2026



RECAP: Foundations of Threshold Signature Scheme (TSS)

- Relies on secure **Multi-Party Computation**
- **Actual workflow:**
 - **Distributed Key Generation:** All parties, say A, B, and C, run a protocol to generate parts of a private key x
 - A, B, and C hold secret shares of x : s_A, s_B, s_C respectively
 - **Signing:** Say A wants to send funds. A messages B and C. Alice computes a partial signature σ_A (using s_A) and so does B and C
 - Partial signatures are combined to obtain σ
 - A b'casts the txn with the combined signature σ

RECAP: More on TSS

- Cheap ($O(1)$ gas cost); Takes place off-chain; high privacy
- TSS implementations — Schnorr's Signatures, ECDSA
- For distributed key generation:
 - Refer to Feldman's Verifiable Secret Shares algorithm



RECAP: The System Models

- Network Synchrony
 - **Synchronous**: node actions are coordinated in rounds (achieved by a common clock)
 - Eg: All nodes bcast in round r , and in round $(r+1)$ all nodes receive the messages
 - **Asynchronous**: No common clock, no bound on n/w latency and drift
 - **Partially synchronous**: No common clock, but upper bound on message delivery
- Fault Model
 - **Crash failure**: Node can stop working abruptly and does not resume
 - **Byzantine failure**: Nodes act arbitrarily — can remain silent and modify messages



RECAP: Distributed Consensus and Failure Models

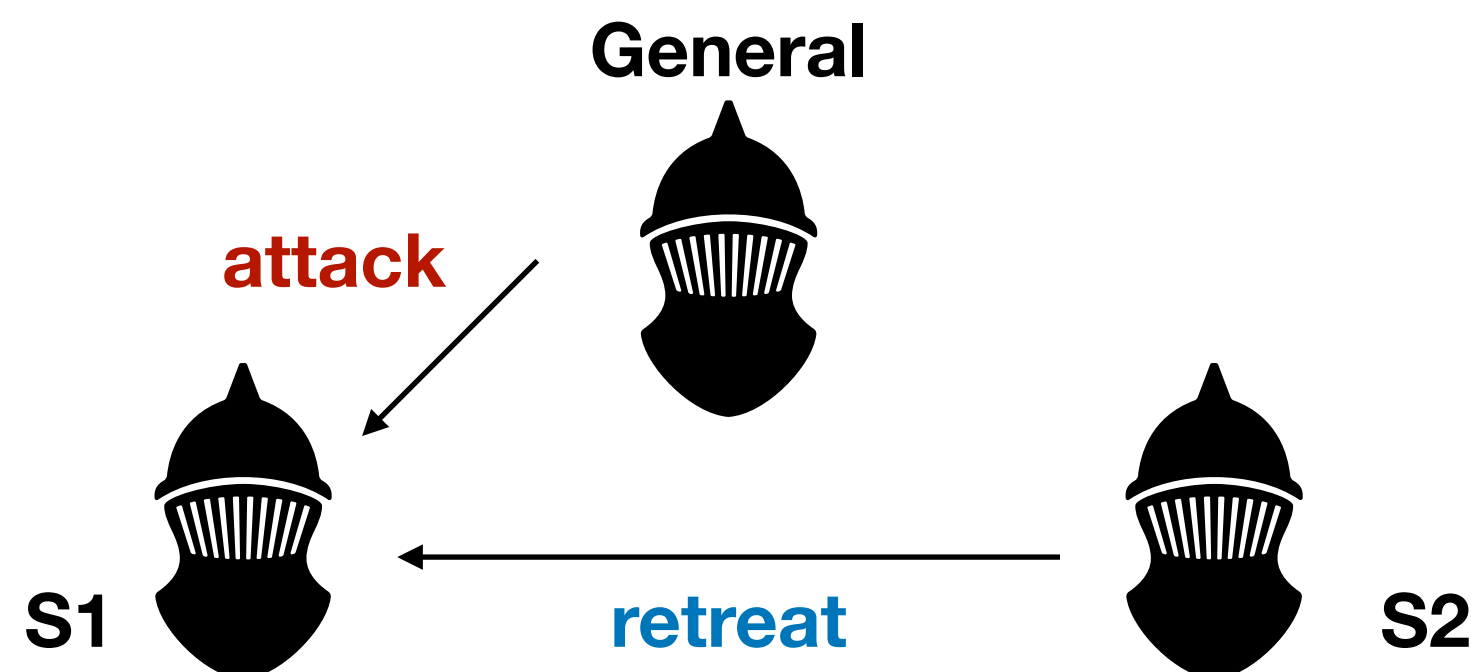
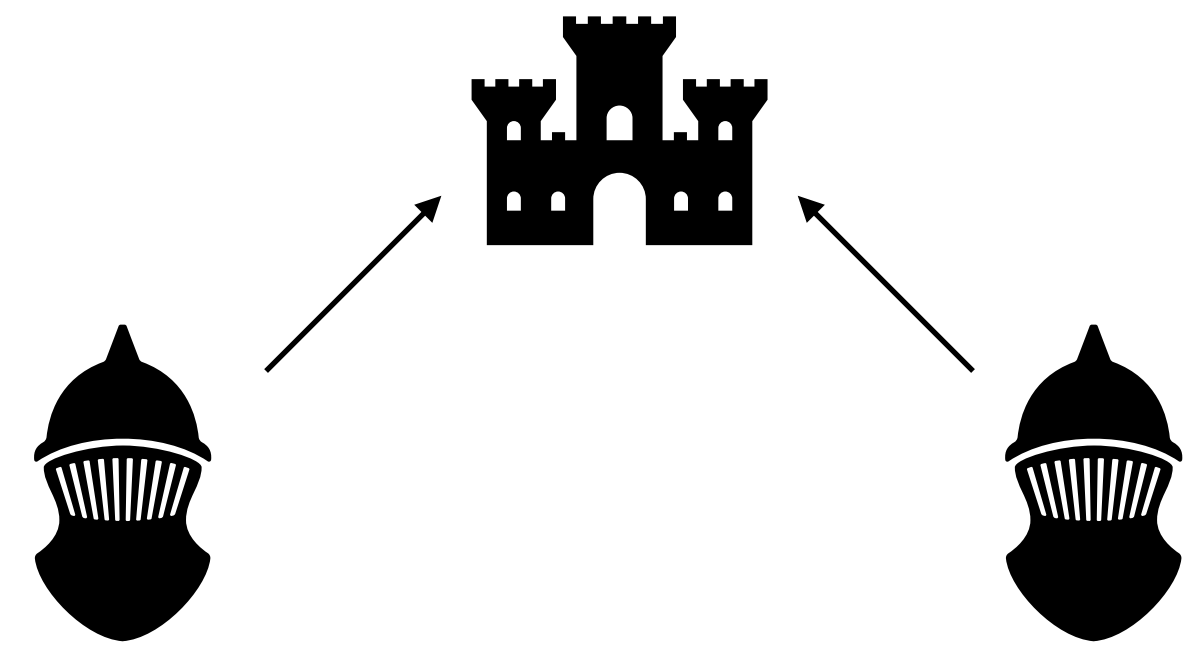
- The consensus problem:
 - **Agreement:** All honest nodes must agree on the same value
 - **Validity:** If all honest nodes have the same input value, then the agreed upon value is the same input value
 - **Termination:** Every honest node must eventually decide a value.

Failure Mode	Synchronous System	Asynchronous system
No failure	Agreement	Agreement
Crash failure	Agreement $f < n$	No agreement
Byzantine failure	Agreement $f \leq n/3$	No agreement

Table: Results on Agreement. n is the total number of processes and f is the number of failure-prone processes.

RECAP: The Byzantine Agreement Problem

- A General commands soldiers
 - If all the soldiers attack then the castle can be won
 - If some retreat then the battle is lost
 - **Byzantine Agreement**: if the loyal (honest) soldiers have a majority agreement
 - **Impossibility Result**: Fisher-Lynch-Paterson, JACM'85
 - What should S1 do?
 - S1 can obey the commander
 - S1 and S2 must obey the same order





RECAP: The Byzantine Agreement Problem

- For m faults, there must at least be $n = 3m + 1$ participants
 - Assumptions: No message loss; Reliable links; **Synchronous** systems
- Complexity: $O(n^m)$ (for **Oral Messaging Protocol**)

Oral Messaging Algorithm

Algorithm 1: $OM(f), f > 0$

```

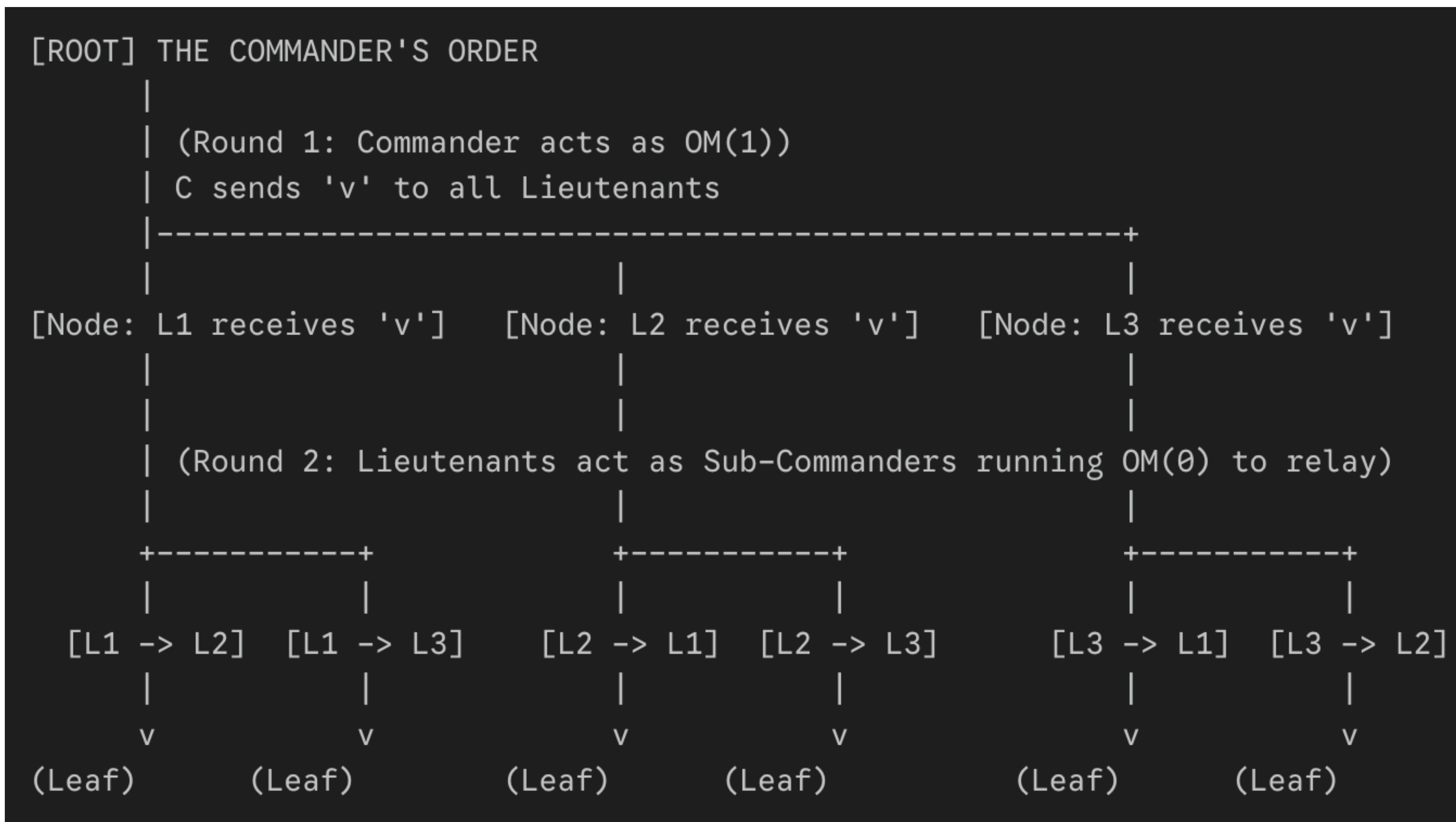
1 Commander sends its value to every lieutenant;
2 for  $i = 1 : N-1$  do
3     Lieutenant  $i$  stores the value received from Commander as  $v_i$ ;
       $v_i = \text{DEFAULT}$  if no value received;
4     Lieutenant  $i$  performs  $OM(f - 1)$  as Commander to send the value
       $v_i$  to the other  $N - 2$  lieutenants;
5 end
6 for  $i = 1 : N-1$  do
7     for  $j = 1 : N-1$  and  $j \neq i$  do
8         Lieutenant  $i$  stores the value received from lieutenant  $j$  as  $v_j$ ;
           $v_j = \text{DEFAULT}$  if no value received;
9     end
10    Lieutenant  $i$  uses  $\text{majority}\{v_1, v_2, \dots, v_{N-1}\}$ ;
11 end

```

- At f failures
- Synchronous n/w
- Commander initiates by calling $OM(f)$
- Default — predetermined — either Retreat or Attack!



Oral Messaging Algorithm



OM Protocol with Signatures

What Problems they do(n't) solve

- Unsigned messages: Faulty nodes can lie what others said
 - Use **digital signatures** — can't be forged
 - When node I passes signed message from node J, the recipient knows that node I did not lie about what node J said!
- With digital signature, the trust problem vanishes
- Scenario:
 - J sends ["attack", signed by J] to I
 - I can't lie to k with the message ["retreat", signed by J] — **Why?**

OM Protocol with Signatures

What Problems they do(n't) solve

- Digital signatures **DO NOT** solve the consensus problem
 - They **fix the trust** problem but **not the timing** problem
 - Since they solve the trust issue — in Synchronous with Byzantine fault model, **we can now have protocol that can tolerate $f < n$ (instead of $f \leq n/3$)**
- **Scenario 2:** Imagine you are waiting from for a signed vote from Node A
 - Node A is a traitor who is deliberately staying silent OR Node A is honest but the n/w is slow
 - **Protocol termination is therefore not guaranteed!**



Consensus under Replication

- **Replication** — *de facto* choice in distributed systems for ensuring **availability**
- **Replication Scheme:**
 - **SMR: State Machine Replication**
 - Consensus protocol is instantiated at each server
 - Goal is to **execute the same order of commands** at each replicated Node
 - Servers run deterministic finite state machines (DFSMs)
 - Replication of DFSM across replicas
 - SMR must guaranteed *safety* — *i.e. all processors execute the same sequence of requests* — and *liveness* — *i.e. all valid requests are executed*

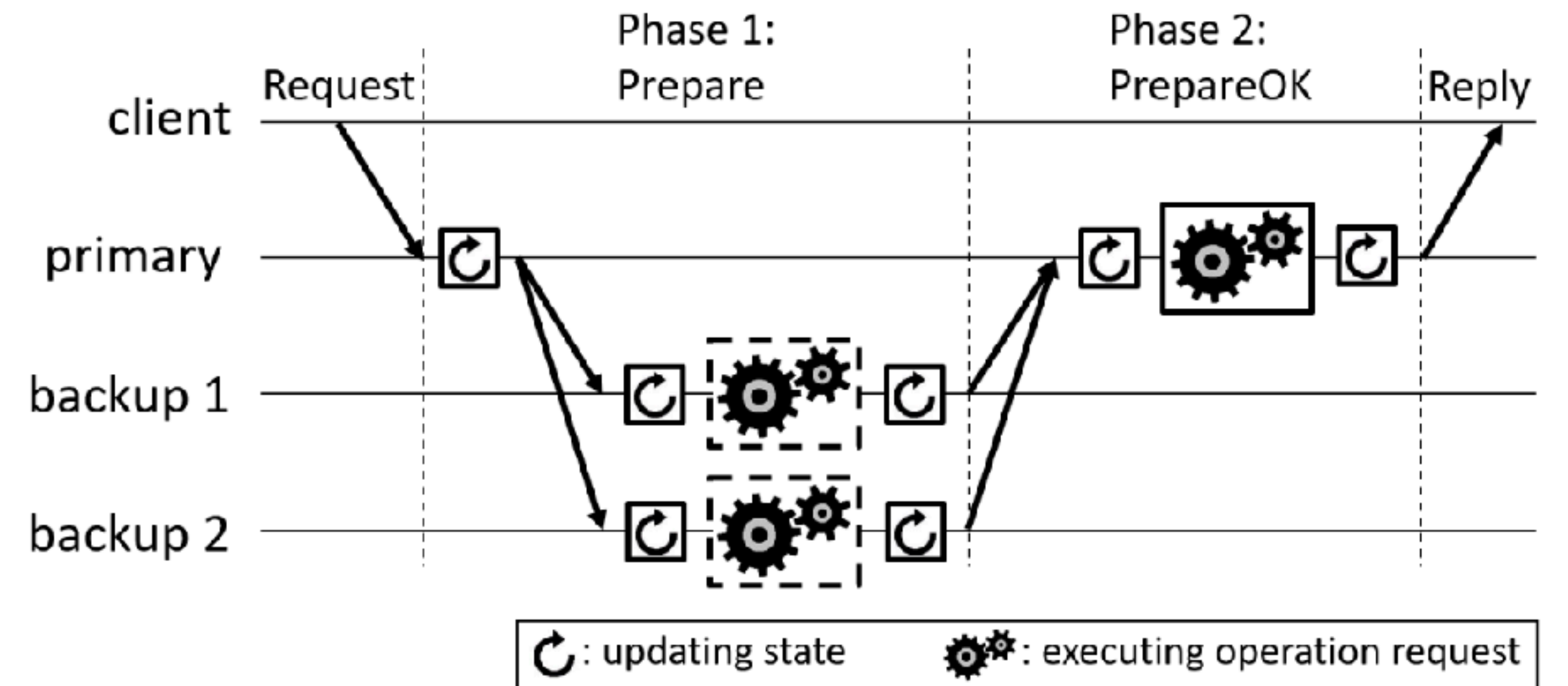


Viewstamped Replication (VSR)

- VSR — proposed by Liskov and Cowling, 2012 — has N replicas where one is *primary* and $N-1$ are backups
- The viewstamp $\langle v, n \rangle$ refers to the current **view number** and **sequence number** of the message **m** that was the most recent request message from a client
 - **View Number:** Determine which replica is the **primary** for the current round
 - **Sequence Number:** A unique number for each client request message **m**
 - **Objective:** Allows for processing of clients' requests in the **correct order**

Viewstamped Replication (VSR)

- VSR has 3 sub-protocols
 - **Normal operation:** when all functioning replicas hold the same view & primary is in good condition
 - **Prepare:** Primary updates n , log, client-table; passes to all replicas as $\langle \text{prepare}, m, v, n, c \rangle$
 - c was updated in the last session
 - Backups execute and updates its state
 - **PrepareOK:** each backup sends message to primary showing its state is up-to-date. After receiving f such messages, the primary executes the requested operation and then updates $c, \text{log}, \text{client} - \text{table}$





Viewstamped Replication (VSR)

- VSR has 3 sub-protocols
 - **Normal operation**
 - **View-Change:** If primary fails
 - Replica updates its status to *ViewChange* and advances to $v + 1$
 - Sends the new view number to other replicas
 - When a replica receives f such messages it sends the message to the replica that will become primary to *DoTheViewChange*
 - The new primary when receives $f + 1$ *DoTheViewChange* messages, it updates its state accordingly. It then sends a *StartView* message to other replicas with the updated *log* and the new view number $v + 1$
 - In the meantime, backup replicas receive the *StartView* message and update their state according to the *log* in the message, and finally change status to *normal*.



Viewstamped Replication (VSR)

- VSR has 3 sub-protocols
 - Normal operation
 - View-Change
 - Recovering: Replica recovers from a crash
 - READING: PBFT advances VSR with Byzantine faults** until it has received at least $f + 1$ *RecoveryResponse* messages
 - Sends *Recovery* msg to all other replicas
 - Each replica responds *RecoveryResponse* with the current v
 - Primary responds with n, c, log in addition to v
 - Then updates the state accordingly.
 - Fault tolerance: $N \geq 2f + 1$
 - Only withstand crash failures!
- VSR has 3 sub-protocols
 - Normal operation
 - View-Change
 - Recovering: Replica recovers from a crash

Complexity Analysis?

BFT Styled Consensus Algorithms

Worth Reading

- **Hedera's Hashgraph Consensus:**
 - Works on a DAG of events rather than a linear chain
 - **Virtual Voting**
 - **Fair ordering**
- **SUI and Aptos (DAG and Pipelined BFTs)**

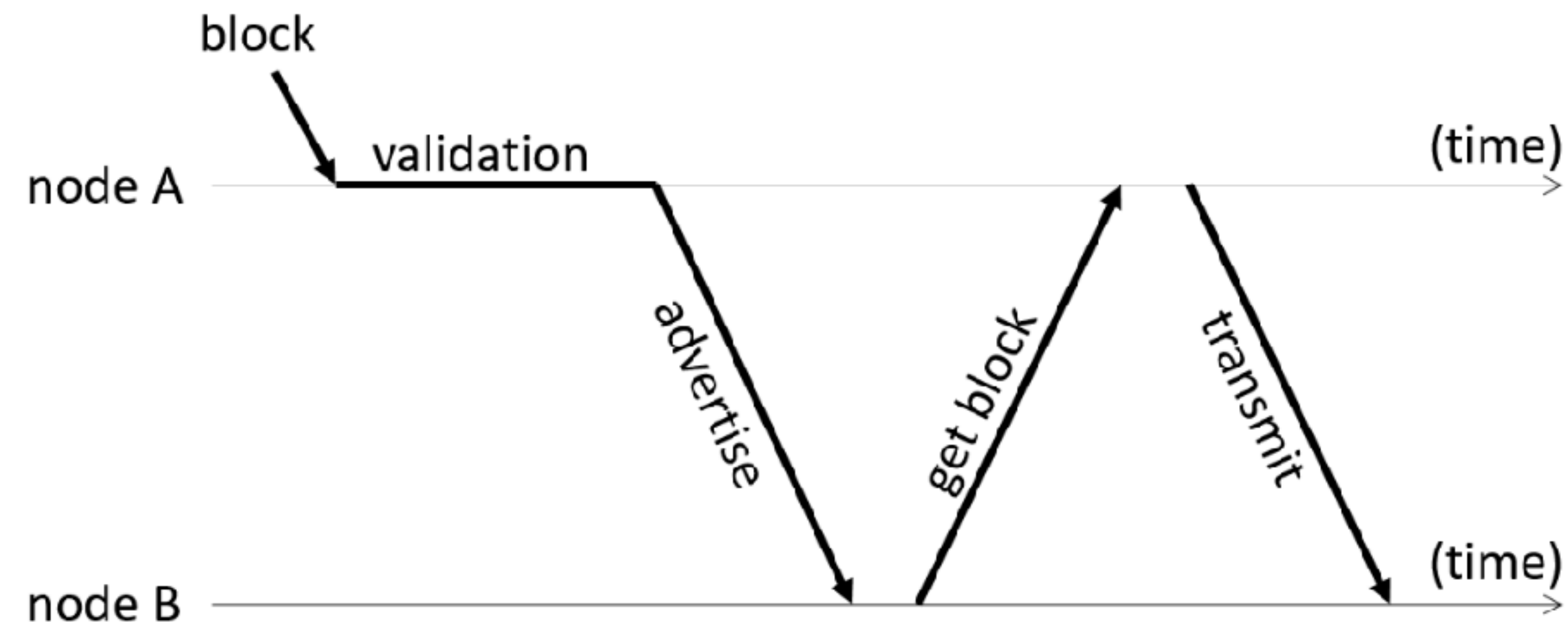


Nakamoto Consensus

- **Network Model:** Peer-to-peer overlay n/w over internet; every node runs the Bitcoin protocol and maintains a replica of Blockchain
 - That is: **asynchronous message-passing system** with bounded transmission delay
 - Additionally, **permissionless access** and best-effort information propagation
 - **Permissionless Access:**
 - Fetch a list of initial peers from known DNS servers; search for new peers by either asking DNS servers or by listening to msgs from other peers; retrieve a blockchain replica from peers and start with the operations

Nakamoto Consensus

- **Information Propagation:**
 - **Reading Assignment:** Christian Decker and Roger Wattenhofer. Information propagation in the bitcoin network.





Nakamoto Consensus Protocol

- Each node runs the **same protocol**
- Each node manages its replica of blockchain **independently**
- **Security:** Depends on majority of nodes being honest
- **Protocol rules:**
 - All newly **received** or **generated** blocks must be bcast to peers in a **timely manner**
 - Blocks and transactions must be validated before bcast
 - **Longest chain is always the desired chain;**



Nakamoto Consensus Protocol

- Longest chain rule:

```
Node A: Block 0 --> Block 1 --> Block 2 --> Block 3 --> Block 4
                                         \
Node B:                               --> Block 3' --> Block 4'
```

- Scenario 1: if Block 4 is mined before Block 4' in the example above, Node B will discard its shorter chain (Block 3' -> Block 4') and switch to the longer chain (Block 3 -> Block 4).
- Scenario 2: If Block 4' is mined before Block 4, and then Block 5 is added to the chain starting from Block 3', all nodes will eventually accept this as the valid chain since it is now the longest.



Nakamoto Consensus Protocol

- **Proof of Work:**
 - Block generation requires inserting a nonce in the header.
 - The hash of the header is less than a particular value
- **Probabilistic Finality:**
 - A block not on the longest chain can be discarded
 - Let $p (\leq 50\%)$ denote the hashing power of an adversary, then the probability of an attacker to catch up from m blocks behind is:
 - $P(\text{catch-up}) = (p/1 - p)^m$